

[부록] 완성된 투표 앱에 기능 추가하기 — 마감 시간 + 결과 그래프

*이 실습은 [1.수업자료(sdd-pocock-skills)-4주.md]에서 만든 투표 앱이 이미 있다는 전제로 진행합니다. 시간이 남는 팀은 이어서, 그렇지 않다면 과제로 진행하세요. 혼자 따라 할 수 있도록 프롬프트와 예상 결과물을 자세히 적어뒀습니다. 사람마다 다른 결과가 있을 수 있으니, 이 파일은 참고만 하시기 바랍니다.

•

왜 이 실습이 의미가 있나

본 수업에서 `/grill-with-docs` 가 남긴 `CONTEXT.md` 와 `docs/adr/` 가 이미 저장소에 있습니다. 이번엔 완전히 새로 인터뷰하는 게 아니라, 그 기록을 시가 다시 읽고 맥락으로 삼은 상태에서 새 결정을 엮게 됩니다. "기록을 남기는 스킴(`/grill-with-docs`)"의 진짜 쓸모는 바로 이런 순간 — 다음 세션, 다음 기능 요청에서 이전 결정을 다시 설명할 필요가 없다는 점 — 에서 드러납니다.

오늘의 기능 요청 (시나리오)

"동아리 회장한테 또 연락이 왔어요. '투표를 계속 열어두니까 사람들이 마감 없이 늘어져서 참여를 안 해요. 마감 시간을 정할 수 있게 해주시고, 결과도 숫자 말고 그래프로 보고 싶어요.'"

실습 1 — `/grill-with-docs`

목표: "마감 시간 + 그래프"라는 모호한 요청을 구체적인 결정들로 바꾸기

입력 프롬프트

`/grill-with-docs`

우리 투표 앱에 마감 시간 기능이랑 결과 그래프를 추가하고 싶어.
동아리 회장이 "투표가 계속 열려있어서 사람들이 참여를 안 한다"고 했어.

진행 중 에이전트가 물어볼 만한 질문

- 마감 시간이 지난 투표는 투표 버튼을 아예 숨길지, 비활성화만 할지?
- 마감 시간이 없는 기존 투표(본 수업에서 만든 투표들)는 어떻게 처리할지?
- 그래프는 막대그래프? 퍼센트로 표시? 투표자 수도 같이 보여줄지?

예상 산출물: `CONTEXT.md` / `docs/adr/` 에 새 결정이 추가됨 (기존 내용 위에 이어서 기록됨)

Decision: 투표 마감 기능

- 마감 시각(`closes_at`)은 선택 입력(`nullable`). 없으면 무기한 진행.
- 마감 시각이 지나면 투표 버튼은 "비활성화 + 안내 문구"로 처리(숨기지 않음).
- 결과 그래프는 옵션별 퍼센트 막대그래프 + 총 투표수 표시.

확인 포인트: 학생 repo의 `CONTEXT.md` 를 열어서, 본 수업 때 남긴 내용 아래에 새 결정이 자연스럽게 이어 붙는지 확인.

실습 2 — /to-spec

입력 프롬프트

/to-spec

예상 산출물: `specs/poll-deadline-and-chart.md`

Spec: 투표 마감 + 결과 그래프

배경

현재 투표는 마감이 없어 참여 동기가 낮음. 결과도 숫자 리스트뿐이라 가독성이 낮음.

요구사항

1. ``polls`` 테이블에 ``closes_at`` (nullable timestampz) 추가
2. 투표 생성 폼에 "마감 시간" 입력 필드 추가 (선택)
3. 투표 페이지: ``closes_at`` 이 지났으면 버튼 비활성화 + "마감된 투표입니다" 표시
4. 결과 페이지: 숫자 리스트 → 옵션별 퍼센트 막대그래프로 교체

성공 기준

- 마감 시간 없이 만든 기존 투표는 동작에 문제 없음
- 마감 후 투표 시도 시 서버에서도 거부됨 (클라이언트만 막지 않음)
- 그래프는 투표 0건일 때도 깨지지 않고 렌더링됨

확인 포인트: "성공 기준"이 테스트로 바로 옮길 수 있을 만큼 구체적인지 점검. ("왜 서버에서도 막아야 하나?" — 클라이언트 검증만으로는 우회 가능하다는 점을 짚어주면 좋은 교육 포인트)

실습 3 — /to-tickets

입력 프롬프트

/to-tickets

예상 산출물: 3개 티켓

- [TICKET 1] DB에 `closes_at` 추가 + 생성 폼에 마감 시간 입력 필드
- [TICKET 2] 투표 페이지: 마감 여부 체크 → 버튼 비활성화 + API에서도 마감된 투표 거부
- [TICKET 3] 결과 페이지: 퍼센트 막대그래프 UI로 교체

확인 포인트: 왜 3개로 나뉘었는지(각각 독립적으로 테스트·배포 가능한 단위인지) 짚기.

실습 4 — /implement

세 티켓을 다 할 시간이 없다면 **티켓 3(그래프 UI)**을 추천합니다 — 눈에 보이는 결과물이 나와서 성취감이 높고, DB 마이그레이션이 없어 실패 확률이 낮습니다. 시간이 넉넉하면 1→2→3 순서로 이어서 진행하세요.

입력 프롬프트 (티켓 3부터 할 경우)

```
/implement
```

TICKET 3: 결과 페이지를 퍼센트 막대그래프로 교체

예상 결과물 (diff 요약)

```
+ components/ResultBar.tsx      (옵셔널 퍼센트 막대 컴포넌트)
~ app/polls/[id]/results/page.tsx (숫자 리스트 → ResultBar 렌더링으로 교체)
+ components/ResultBar.test.tsx  (투표 0건 / 1건 / 동률 케이스 테스트)
```

/implement 는 TDD로 동작합니다 — 실패하는 테스트를 먼저 작성 → 통과하는 최소 코드 작성 → 리팩터링 순서로 진행됩니다. 터미널에 뜨는 테스트 결과(빨강→초록)를 지켜보세요.

확인 포인트(결과물 체크리스트)

- `npm run test` 통과 화면 캡처
- `npm run dev` 로 실제 브라우저에서 막대그래프가 퍼센트로 잘 뜨는지
- 투표가 0건인 투표를 결과 페이지에서 열었을 때 에러 없이 "아직 투표 없음" 같은 문구가 뜨는지 (성공 기준에 있던 엠티케이스)

티켓 1, 2를 이어서 할 경우

```
/implement
```

TICKET 1: DB에 `closes_at` 추가 + 생성 폼에 마감 시간 입력 필드

```
/implement
```

TICKET 2: 투표 페이지 마감 여부 체크 + API에서도 마감된 투표 거부

티켓 1은 `db/schema.sql` 에 `alter table polls add column closes_at timestampz;` 같은 마이그레이션 SQL이 추가로 생성됩니다 — 본 수업 TICKET 1 때처럼, 이 SQL을 Neon SQL Editor에서 직접 실행해야 실제 컬럼이 추가됩니다.

남은 티켓을 AFK로 돌려보기: 시간이 부족하면, 팀별로 티켓 1개씩을 골라 /implement 를 사람 개입 없이(AFK) 돌려보고 결과만 나중에 같이 리뷰하는 것도 좋은 경험이 됩니다. 다만 AFK는 애매한 지점에서 에이전트가 스스로 가정하고 진행할 수 있으니, 다음 리뷰 단계에서 그 가정이 맞았는지 꼭 확인하세요.

실습 5 — /code-review

입력 프롬프트

```
/code-review
```

예상 산출물

Code Review: TICKET 3

- ✅ 성공 기준 충족: 0건일 때도 렌더링 정상
- ✅ 테스트 3개 모두 그린
- ⚠️ 제안: 퍼센트 계산 시 소수점 반올림 방식이 명시 안 됨 (반올림 vs 버림)
- ⚠️ 제안: 막대그래프 색상이 옵션 5개 이상일 때 구분이 잘 안 될 수 있음

확인 포인트: "⚠️ 제안"이 나오면 그걸 바로 반영할지, 다음 과제로 넘길지 판단하게 해보기 — 사람이 판단력/taste를 엮는 지점.

배포

본 수업 때 이미 Vercel에 배포했다면, 오늘 바뀐 코드는 `git push` 만 하면 자동으로 재배포됩니다. 아직 배포한 적이 없다면 [[class-note-sdd-pocock-skills.md](#)]의 "부록 — Vercel 배포 가이드"를 그대로 따라 하세요 (테이블 변경사항이 있다면 Neon SQL Editor에서 `alter table` 문도 함께 실행해야 합니다).